

TinyLlama Medusa Optimization Report Draft

This document summarizes the TinyLlama-focused PDC project results. It is meant as a reusable evidence file for the final report/presentation, not as polished prose.

Vicuna 7B results are intentionally excluded from the main conclusion because the Colab 8-bit Vicuna optimization runs were not correctness-stable enough to support the final claim. The TinyLlama results below are the successful local optimization path.

1. Problem And Gap

The base LLM inference bottleneck is autoregressive decoding: the model generates one token per forward pass, so generation latency is dominated by a sequential loop. Speculative decoding and speculative sampling address this by drafting multiple tokens and verifying them with the target model while preserving the target behavior under their acceptance rules (Leviathan et al., 2023; Chen et al., 2023). Medusa adapts the same multi-token verification idea without a separate draft model: it adds multiple decoding heads that speculate future tokens and uses tree attention to verify candidate paths in parallel (Cai et al., 2024).

The gap we found inside Medusa is that it shifts the bottleneck from pure sequential decoding to candidate-tree verification. Medusa can propose many future tokens, but it still has to:

- materialize candidate paths,
- run tree verification,
- compute acceptance from tree logits,
- update the KV cache for accepted tokens,
- and sometimes verify many branches to accept only a few tokens.

So the optimization target became: keep Medusa's speculative benefit while reducing verifier, tree, and KV-cache overhead.

2. Main Conclusions

- TinyLlama Medusa improves throughput over plain autoregressive TinyLlama. In the early 10-prompt suite, plain TinyLlama averaged 80.74 TPS, while Medusa averaged 102.53 TPS, a 1.27x throughput gain.
- In a smaller local 3-prompt baseline comparison, plain simple_base averaged 73.66 TPS, while medusa_base averaged 128.33 TPS, a 1.74x gain. That run also shows Medusa's extra memory cost: peak allocation increased from about 2124 MB to 2843 MB.
- The strongest implementation optimization was the greedy full-tree verifier + Triton KV-copy path. It preserved prefix match and reached 123.14 TPS vs 105.75 TPS for Medusa base, a 1.16x speedup over Medusa with about 104 MB lower peak allocation.
- The later current old-head architecture reached about 1.10x over the already-optimized Medusa baseline, with about 147 MB lower peak allocation.
- On the trained/self-distilled TinyLlama heads, the stable final optimization gave about 1.05x over Medusa base, prefix match 1.0, and about 119 MB lower peak allocation.
- The compact "Turbo 25 nodes vs Medusa 64 nodes" experiment succeeded: turbo_fast_24 verified about 25 nodes/step instead of 64, achieved about 1.24x speedup over Medusa base, preserved prefix match, and reduced peak allocation by about 150 MB.
- Aggressive QJL/TurboVQ/pruning paths were explored but are not the main final claim because they were slower, unstable, or only near break-even end-to-end on TinyLlama.

3. What Our Optimized Medusa Added Over Medusa Base

The optimized implementation did not replace Medusa's core idea. It kept the same basic speculative-decoding contract: Medusa heads propose future tokens, a candidate tree is verified, and the longest accepted prefix is appended, matching the Medusa tree-attention design (Cai et al., 2024). The changes targeted the cost of making that contract run on GPU. This systems motivation is aligned with IO-aware attention work such as FlashAttention: reducing HBM traffic and fusing GPU work can matter as much as the raw algorithmic FLOP count (Dao et al., 2022).

Area	Medusa Base	Optimized Medusa
Verification	Generic full-tree acceptance path	Greedy full-tree verifier fast path for <code>temperature=0</code>
Acceptance work	More path/depth/vocab gather and comparison tensors	Triton fused candidate comparison, cumulative prefix acceptance, and best-path selection
KV update	Broader cache-copy/update work after acceptance	Triton selected-position KV-cache copy for only the accepted suffix tokens
Tree buffers	Rebuilds or rematerializes more tree state	Cached fixed tree masks and reusable tree/layout buffers
Logits/head work	Computes more Medusa-head/logit state than needed in some paths	Lazy Medusa-head/logit computation only where needed, including accepted-node update paths
Token buffers	Repeated <code>torch.cat</code> /output rebuilding in streaming paths	Preallocated token/output buffers to reduce allocation and concatenation overhead
Tree size option	Full 63-choice / 64-node tree	Output-preserving full-tree fast path, plus optional smaller-tree mode such as <code>turbo_fast_24</code> that verifies about 25 nodes

The safest optimization is the full-tree fast verifier: it keeps the Medusa tree coverage the same and reduces verifier/KV overhead. The smaller-tree path is a bottleneck demonstration: it shows that, when the full tree is not buying enough accepted tokens, reducing verification from 64 nodes to about 25 nodes can improve speed while preserving prefix match in the compact TinyLlama run.

4. Training And Head Quality

We trained/self-distilled TinyLlama Medusa heads with the base model frozen, following the Medusa-1 style idea of adding heads on top of a frozen backbone and using self-distillation when task data is limited (Cai et al., 2024). The best saved self-distilled TinyLlama checkpoint reached:

Metric	Head 1	Head 2	Head 3	Head 4
Top-1	0.399	0.220	0.152	0.118
Top-5	0.639	0.421	0.329	0.283
Top-10	0.722	0.513	0.416	0.369

Source: `artifacts/benchmarks/model_metrics/medusa_tinyllama_heads_selfdistill_3060_training_metrics.json`.

This matters because Medusa speedup depends on accepted tokens per decoding step. In the stable TinyLlama mixed-suite runs, accepted tokens per step were typically around 2.2, which is enough to beat plain autoregressive decoding but also shows why verification overhead matters: many candidate nodes are still checked for roughly two accepted tokens.

5. Successful Optimization Results

5.1 Plain TinyLlama vs Medusa

Experiment	Baseline	Optimized	TPS Baseline	TPS Optimized	Speedup	Notes
Early 10-prompt suite	Plain TinyLlama	Medusa	80.740	102.533	1.270x	Source: <code>artifacts/benchmarks/medusa/benchmark_results.csv</code>
Local 3-prompt suite	<code>simple_base</code>	<code>medusa_base</code>	73.662	128.327	1.742x	Source: <code>artifacts/benchmarks/medusa/turbo_benchmark_results.csv</code>

Interpretation: Medusa successfully reduces the base model's sequential decoding bottleneck. However, Medusa uses more memory than plain base because it adds heads and verifies tree candidates.

5.2 Greedy Full-Tree Verifier + Triton KV-Copy

Mode	TPS	Prefix Match	Peak Alloc
<code>medusa_base</code>	105.746	1.000	2854.5 MB
<code>turbo_force_fast_fulltree</code>	123.140	1.000	2750.3 MB

Speedup over Medusa base: 1.164x.

Memory reduction: about 104.3 MB, or about 3.7% lower peak allocation.

Source: artifacts/benchmarks/medusa/tinyllama_fused_verifier_kvcopy_benchmark.csv .

What changed:

- Used a greedy verifier fast path for temperature=0 .
- Avoided expensive duplicated path/depth/vocab acceptance gathers.
- Fused candidate comparison, cumulative prefix acceptance, and best-path selection with Triton.
- Added dense contiguous FP16 KV-cache selected-position copy with Triton.
- Skipped unnecessary root self-copy.

This is the cleanest “we optimized Medusa itself” result because it keeps the full tree and preserves output prefix match.

5.3 Current Old-Head Best Architecture

Mode	TPS	Prefix Match	Peak Alloc
medusa_base_stream	118.051	1.000	2851.4 MB
turbo_best_stream	129.649	1.000	2704.5 MB
medusa_base_nonstream	118.504	1.000	2851.4 MB
turbo_best_nonstream	130.230	1.000	2704.5 MB

Speedup over Medusa base:

- Streaming: 1.098x .
- Non-streaming: 1.099x .

Memory reduction: about 147 MB , or about 5.2% .

Source: artifacts/benchmarks/medusa/tinyllama_turbo_best_old_heads_benchmark_fresh.csv .

Additional implementation improvements in this path:

- Lazily computed Medusa-head logits only for the accepted tree node.
- Cached fixed full-tree attention masks.
- Used last-token prompt logits for Turbo prefill.
- Added optional non-streaming output mode to avoid repeated output work.

5.4 Trained/Self-Distilled TinyLlama Heads

Mode	TPS	Prefix Match	Peak Alloc
medusa_base	70.342	1.000	2350.6 MB
turbo_force_fast_fulltree	74.026	1.000	2231.1 MB

Speedup over Medusa base: 1.052x .

Memory reduction: about 119.5 MB , or about 5.1% .

Source: artifacts/benchmarks/medusa/tinyllama_trained_heads_fused_benchmark.csv .

This is a good conservative final number for the trained-head path: smaller than the older 1.16x result, but stable and correctness-preserving.

5.5 Self-Distilled Step 1200/1500 Runs

Run	Mode	TPS	Speedup vs Medusa	Accepted Tokens/Step	Verified Nodes/Step	Prefix	Peak Alloc
Step 1200 short eval	medusa_base	120.033	1.000x	2.100	25.0	1.000	2754.8 MB
Step 1200 short eval	turbo_best_full_tree	134.492	1.120x	2.100	25.0	1.000	2701.5 MB
Step 1500 mixed	medusa_base	133.736	1.000x	2.212	25.0	1.000	2748.3 MB
Step 1500 mixed	turbo_fast_24	139.468	1.043x	2.207	25.0	0.993	2696.2 MB

Sources:

- artifacts/benchmarks/medusa/local_selfdistill_3060_step1200_eval.csv
- artifacts/benchmarks/medusa/local_selfdistill_3060_step1500_mixed.csv

Interpretation: after the baseline itself became more optimized, the extra Turbo gain settled around 1.04x to 1.12x , with lower memory and near-identical outputs.

5.6 Fixed-Token Choice-Tree Sweep

The fixed-token sweep is useful because it avoids misleading speedups from one mode generating fewer tokens.

Mode	TPS	Speedup	Accepted Tokens/Step	Verified Nodes/Step	Prefix	Peak Alloc
medusa_base	135.315	1.000x	2.249	25.0	1.000	2748.3 MB
turbo_fast_20	138.972	1.027x	2.174	21.0	1.000	2695.7 MB
turbo_fast_24	142.430	1.052x	2.249	25.0	1.000	2695.9 MB
turbo_adaptive_16_24	138.803	1.026x	2.168	18.5	1.000	2695.6 MB
turbo_adaptive_20_32	138.339	1.022x	2.199	23.1	1.000	2696.1 MB

Source: artifacts/benchmarks/medusa/local_choice_tree_fixed_tokens_step1500_compact.csv .

Best safe result in this table: turbo_fast_24 , 1.052x , prefix 1.0 , about 52 MB lower peak allocation.

6. Turbo 25 Nodes vs Medusa 64 Nodes

This is the result to use when discussing the Medusa tree-verification bottleneck most directly.

In this compact experiment, medusa_base used the full 63-choice / 64-node tree, while turbo_fast_24 used a 24-choice tree, which verifies about 25 nodes including the root.

Mode	TPS	Speedup	Accepted Tokens/Step	Verified Nodes/Step	Prefix	Peak Alloc
medusa_base	72.003	1.000x	1.300	64.0	1.000	2347.3 MB
turbo_best_full_tree	81.590	1.134x	1.300	64.0	1.000	2200.5 MB
turbo_fast_24	89.237	1.239x	1.271	25.0	1.000	2197.5 MB

Source: artifacts/benchmarks/medusa/local_4head_fused_lm_head_argmax_final.csv .

Key points:

- Verified nodes dropped from 64.0 to 25.0 , about a 61% reduction in verifier tree nodes.
- TPS improved from 72.003 to 89.237 , a 1.239x speedup over Medusa base.
- Prefix match stayed 1.000 .
- Peak allocation dropped by about 149.8 MB , about 6.4% .

This supports the report claim that a major Medusa bottleneck is over-verification: checking a large tree can cost more than the additional accepted tokens are worth. A smaller or faster verifier can be better when head quality or prompt structure does not justify the full tree.

Caveat: this was a compact two-prompt old-head experiment. It is a strong bottleneck demonstration, but the broader mixed-suite trained-head number is closer to 1.05x .

7. Same 64-Node Tree, Faster Full-Tree Verifier

To separate “smaller tree” from “faster verifier implementation”, we also tested a full-tree fast verifier.

Mode	TPS	Verified Nodes/Step	Prefix	Peak Alloc
full4_base	98.839	64.0	1.000	2851.3 MB
full4_fast	120.921	64.0	1.000	2704.5 MB

Speedup over full-tree Medusa base: 1.223x .

Source: artifacts/benchmarks/medusa/tinyllama_more_heads_local_compare.csv .

Interpretation: even without reducing tree size, optimizing verifier implementation and memory movement gives a substantial speedup. This supports the claim that Medusa's verification implementation overhead, not only tree size, was a real bottleneck.

8. Memory Results

Experiment	Baseline Peak Alloc	Optimized Peak Alloc	Reduction
Fused verifier + KV-copy	2854.5 MB	2750.3 MB	104.3 MB
Current old-head streaming	2851.4 MB	2704.5 MB	147.0 MB
Trained self-distilled heads	2350.6 MB	2231.1 MB	119.5 MB
Step1500 fixed-token <code>turbo_fast_24</code>	2748.3 MB	2695.9 MB	52.4 MB
Turbo 25 nodes vs Medusa 64 nodes	2347.3 MB	2197.5 MB	149.8 MB

The memory reductions came mainly from:

- avoiding duplicated verifier tensors,
- reducing path/depth/vocab materialization,
- caching masks and buffers,
- avoiding repeated `torch.cat` output/input rebuilding,
- and using fused selected-position KV-copy instead of broader copy/update work.

8.1 Quick Roofline-Style Estimate

This is a roofline-style estimate, not a formal Nsight Compute roofline. We did not collect hardware-counter FLOPs or HBM bytes. Instead, we combined the TinyLlama config with benchmark counters to estimate whether the successful runs looked KV-cache-bandwidth dominated.

Assumptions:

- TinyLlama config: 22 layers, hidden size 2048, intermediate size 5632, 32 attention heads, 4 KV heads.
- Estimated parameter count: about 1.10B, so FP16/BF16 weight footprint is about 2.20 GB.
- Estimated linear-layer cost: about 2.20 GFLOPs per verified tree node.
- Approximate arithmetic intensity for a verifier step is roughly `verified_nodes_per_step` FLOPs/byte if model weights are streamed once and reused across the verified tree nodes.

Experiment	Mode	TPS	Accepted/Step	Nodes/Step	Est. AI	Est. Achieved TFLOP/s	Est. Weight BW	FP16 KV Est.	KV / Peak Alloc
64 vs 25 compact	<code>medusa_base</code>	72.003	1.300	64.0	64 FLOP/B	7.80	121.8 GB/s	1.69 MB	0.072%
64 vs 25 compact	<code>turbo_best_full_tree</code>	81.590	1.300	64.0	64 FLOP/B	8.84	138.1 GB/s	1.69 MB	0.077%
64 vs 25 compact	<code>turbo_fast_24</code>	89.237	1.271	25.0	25 FLOP/B	3.86	154.4 GB/s	1.67 MB	0.076%
Fixed-token compact	<code>medusa_base</code>	135.315	2.249	25.0	25 FLOP/B	3.31	132.4 GB/s	3.14 MB	0.114%
Fixed-token compact	<code>turbo_fast_24</code>	142.430	2.249	25.0	25 FLOP/B	3.48	139.3 GB/s	3.14 MB	0.117%

Interpretation:

- The TinyLlama KV cache estimate is tiny relative to total allocation: roughly 1.7-3.1 MB, or only about 0.07%-0.12% of peak allocation in these runs.
- This supports the QJL/TurboQuant conclusion: on these short-context TinyLlama prompts, compressing KV cache could not remove the dominant end-to-end bottleneck because KV cache traffic was not large enough.
- The 64-node tree has higher estimated arithmetic intensity, but it spends more verifier work for similar accepted tokens. `turbo_fast_24` wins by doing less tree-verification work, not by improving KV-cache compression.
- Estimated achieved TFLOP/s and weight-bandwidth numbers are well below typical GPU theoretical peaks, so the bottleneck is likely a mixture of verifier tensor materialization, small/irregular tree shapes, launch overhead, KV-copy/update work, and non-ideal GPU utilization rather than a pure peak-compute roof.

For a formal roofline, we would need Nsight Compute counters for achieved FLOPs and HBM/DRAM bytes, then compare against the actual GPU's peak FLOP/s and memory bandwidth.

9. Accuracy And Correctness Proxy

Most successful TinyLlama optimization runs used prefix match versus Medusa base as the correctness proxy.

Successful final/report-safe modes:

- `turbo_force_fast_fulltree` : prefix 1.000 .
- `turbo_best_stream` / `turbo_best_nonstream` : prefix 1.000 .
- trained-head `turbo_force_fast_fulltree` : prefix 1.000 .
- fixed-token `turbo_fast_24` : prefix 1.000 .
- Turbo 25-node compact experiment: prefix 1.000 .

Near-safe but weaker:

- Step1500 mixed `turbo_fast_24` : prefix 0.993 .

Not report-safe as correctness-preserving:

- more aggressive tree sizes such as `turbo_fast_28` and above sometimes improved TPS but reduced prefix match.
- QJL pruning and strict TurboVQ modes were not consistently stable end-to-end.

10. Triton And Kernel-Level Changes

The project included several Triton/kernel-level improvements. The kernel story follows the same broad systems lesson as FlashAttention: exact model behavior can still be accelerated by reducing memory movement, avoiding unnecessary materialization, and organizing GPU work around the memory hierarchy (Dao et al., 2022). The most important changes for the final TinyLlama story were:

10.1 Fused Greedy Verifier

- Fused candidate equality checks, cumulative acceptance, and best path selection.
- Kept expensive vocabulary argmax in optimized PyTorch because a one-kernel full-vocab verifier was slower for 32k-vocab LLM shapes.
- Reduced verifier-chain microbenchmark time on TinyLlama-shaped trees from about 134 us to about 30 us .

10.2 Triton KV-Cache Selected Copy

- Added selected-position copy for accepted suffix positions.
- Avoided copying unnecessary root/self positions.
- Helped reduce end-to-end peak memory and verifier update overhead.

11. QJL And TurboQuant: What We Implemented And Why It Did Not Win End-To-End

QJL and TurboQuant were real implementation work, not just placeholders. They are based on a line of KV-cache compression and serving work showing that KV memory grows with context length and can become a major bottleneck in long-context or high-batch inference (Kwon et al., 2023; Liu et al., 2024; Hooper et al., 2024). QJL motivates 1-bit JL sign sketches for low-overhead KV-cache inner-product estimation, while PolarQuant and TurboQuant motivate random transformations, scalar/vector quantization, and residual QJL correction for compressed KV caches (Zandieh et al., 2024; Han et al., 2025; Zandieh et al., 2025). These paths did not become the final TinyLlama speedup path because the TinyLlama benchmark setting had short contexts, so KV cache traffic was not the dominant end-to-end bottleneck.

11.1 What We Implemented

Component	Implementation
QJL path scoring	Stored/used 1-bit sign sketches for approximate query-key and candidate-path scoring, inspired by QJL's sign-sketch inner-product estimator (Zandieh et al., 2024).
QJL pruning planner	Used approximate pass-1 scores to prefilter candidate paths, while keeping ambiguous cases on full-tree verification for correctness.
Node-budget pruning	Controlled pruning by actual unique verifier nodes, not only number of selected paths.
TurboQuant/TurboVQ KV cache	Added random rotation, Lloyd-Max scalar codebooks, per-vector normalization, residual reconstruction, residual norm storage, and packed 1-bit residual QJL signs, following the TurboQuant/PolarQuant direction for compressed KV representations (Han et al., 2025; Zandieh et al., 2025).
Triton QJL kernels	Fused QJL sign-cache gather, dot product, scaling, masking, and path reduction.
Triton pruned materialization	Fused selected-node gather and padded path-candidate gather for pruned Medusa layouts.
Triton compressed attention	Added direct compressed-cache attention for strict compressed KV paths.
Triton TurboVQ append	Fused rotation, codebook scan, code-index writes, residual sketch projection, sign packing, and residual norm writes for the decode append path.
Hybrid hot-window attention	Kept recent KV exactly in FP16/BF16 while storing older KV in compressed TurboVQ form.

11.2 Microbenchmark Wins

Kernel/Microbenchmark	Result
Fused TurboVQ decode attention, head_dim 64, kv_len 256	58.21 us vs 435.17 us , 7.48x vs PyTorch reference
Fused TurboVQ decode attention, head_dim 128, kv_len 256	85.58 us vs 445.74 us , 5.21x vs PyTorch reference
Fused append + attention, 8 heads, head_dim 64, kv_len 256	85.46 us vs 359.16 us , about 4.20x
Fused append + attention, 8 heads, head_dim 128, kv_len 256	86.89 us vs 360.64 us , about 4.15x
Hybrid TurboVQ, 8 heads, head_dim 64, kv_len 512	63.30 us vs 886.14 us , 14.00x reference speedup
Hybrid TurboVQ, 32 heads, head_dim 128, kv_len 1024	731.22 us vs 1865.66 us , 2.55x reference speedup

Sources:

- artifacts/benchmarks/medusa/turbo_vq_decode_optimized_microbench.csv
- artifacts/benchmarks/medusa/turbo_vq_append_optimized_microbench.csv
- artifacts/benchmarks/medusa/turbo_vq_hybrid_microbench.csv

11.3 Why It Did Not Become The Final TinyLlama Speedup

The main issue was a workload mismatch. Prior KV-cache systems and quantization work targets settings where the KV cache is large enough to dominate memory capacity, memory bandwidth, batching, or long-context serving behavior (Kwon et al., 2023; Liu et al., 2024; Hooper et al., 2024). TurboQuant and QJL are most useful in that regime because their compression/scoring work pays off when KV traffic is a large fraction of runtime (Zandieh et al., 2024; Zandieh et al., 2025). Our TinyLlama prompts used a small context window / short-context runs, so the estimated FP16 KV cache was often only a few MB, while model/verifier allocations were in the multi-GB range. For example, the fixed-token TinyLlama sweep had peak allocation around 2.7 GB , while the estimated FP16 KV cache was only about 3-5 MB .

Because KV cache was not the dominant cost, compressing KV saved little end-to-end time. The compressed paths also added their own overhead:

- QJL added pass-1 scoring and planning work before verification.
- TurboQuant added quantization, dequantization, random rotation, residual reconstruction, and residual-QJL correction.
- Safe pruning often fell back to full-tree verification to preserve prefix correctness, so pass-1 work became extra overhead.
- For Medusa tree blocks, PyTorch/vendor SDPA over a regular FP16 cache was often faster than our custom compressed-cache path.
- Strict compressed KV sometimes caused prefix drift, so it was not report-safe as a correctness-preserving optimization.

Conclusion: QJL/TurboQuant is promising for long-context workloads where KV cache is large enough to dominate bandwidth and memory, consistent with the motivation of recent KV-cache serving and quantization papers, but it was not the right final speedup path for short-context TinyLlama (Kwon et al., 2023; Zandieh et al., 2024; Zandieh et al., 2025).

12. Experiments That Did Not Become Final Claims

Experiment	Outcome	Why Not Final
turbo_prune_only early local benchmark	119.73 TPS vs 128.33 TPS for Medusa base, prefix 1.0	Correct but slower than Medusa in that run.
Adaptive/QJL pruning	Best safe raw-score runs were near break-even, e.g. about 0.966x-0.974x of Medusa base with prefix 1.0	QJL reduced some work, but pass-1 scoring and fallback overhead outweighed savings on short contexts.
Node-budget pruning on trained heads	Near break-even with prefix preserved	Did not beat the full-tree fast verifier, which was simpler and faster.
Larger fixed trees such as turbo_fast_28	Higher TPS in some sweeps	Prefix match fell below report-safe level.
Strict TurboVQ compressed generation	Strong microbenchmarks and useful kernels	End-to-end generation was slower and sometimes had prefix drift.
Vicuna 7B Colab 8-bit optimization	Medusa itself got about 1.6x vs plain Vicuna	Optimized/pruned modes had poor prefix match, so excluded from final TinyLlama claim.

This is still valuable because it narrows the final story: our successful optimization was not arbitrary pruning or quantization; it was reducing verifier/KV overhead while preserving the Medusa acceptance behavior. QJL/TurboQuant should be presented as technically meaningful exploratory work whose expected advantage requires longer contexts than the final TinyLlama benchmark used.

13. Recommended Final Report Claim

Literature context: speculative decoding and Medusa explain why multi-token verification can reduce autoregressive decoding steps, while IO-aware kernels and KV-cache work explain why memory movement and context length determine whether verifier overhead or KV compression is the right optimization target (Leviathan et al., 2023; Chen et al., 2023; Cai et al., 2024; Dao et al., 2022; Kwon et al., 2023; Zandieh et al., 2024; Zandieh et al., 2025).

Suggested wording:

We found that after Medusa removes the base model's one-token-at-a-time bottleneck, the next bottleneck is candidate-tree verification and KV-cache update overhead. We optimized this stage using a greedy verifier fast path, Triton-assisted acceptance/KV-copy kernels, cached tree buffers, and tuned smaller-tree variants. On TinyLlama, Medusa improved throughput over plain autoregressive decoding by about 1.27x to 1.74x depending on the benchmark suite. Our best correctness-preserving Medusa optimization improved over Medusa base by up to 1.16x in the fused verifier/KV-copy run, while the stable trained-head result achieved about 1.05x with prefix match 1.0 and about 119 MB lower peak allocation. In a compact full-tree comparison, reducing verification from 64 nodes to 25 nodes improved throughput by about 1.24x while preserving prefix match, directly demonstrating that Medusa's verification tree can become a bottleneck. We also implemented QJL/TurboQuant KV-cache compression and pruning kernels, but for this TinyLlama workload the context window was too small for KV-cache compression to dominate end-to-end runtime, so those paths remained exploratory rather than the final speedup.

Conservative final number to cite:

- Medusa over plain TinyLlama: 1.27x on the 10-prompt suite.
- Optimized verifier over Medusa base: 1.05x stable trained-head result.
- Best implementation optimization over Medusa base: 1.16x fused verifier/KV-copy result.
- Bottleneck demonstration: 1.24x for Turbo 25-node verifier vs Medusa 64-node verifier.

14. Source Files And Repo Pointers

Key result CSVs:

- `artifacts/benchmarks/medusa/benchmark_results.csv`
- `artifacts/benchmarks/medusa/turbo_benchmark_results.csv`
- `artifacts/benchmarks/medusa/tinylama_fused_verifier_kvcopy_benchmark.csv`
- `artifacts/benchmarks/medusa/tinylama_turbo_best_old_heads_benchmark_fresh.csv`
- `artifacts/benchmarks/medusa/tinylama_trained_heads_fused_benchmark.csv`
- `artifacts/benchmarks/medusa/local_choice_tree_fixed_tokens_step1500_compact.csv`
- `artifacts/benchmarks/medusa/local_4head_fused_lm_head_argmax_final.csv`
- `artifacts/benchmarks/medusa/tinylama_more_heads_local_compare.csv`
- `artifacts/benchmarks/medusa/tinylama_adaptive_prune_rawscore_benchmark.csv`
- `artifacts/benchmarks/medusa/turbo_vq_decode_optimized_microbench.csv`
- `artifacts/benchmarks/medusa/turbo_vq_append_optimized_microbench.csv`
- `artifacts/benchmarks/medusa/turbo_vq_append_optimized_generation_repeat.csv`
- `artifacts/benchmarks/medusa/turbo_vq_hybrid_microbench.csv`

Key code areas:

- `Medusa/medusa/model/medusa_model.py` : Medusa generation loop, Turbo verifier modes, pruning/planning integration.
- `Medusa/medusa/model/utils.py` : candidate generation, tree buffers, acceptance utilities.
- `Medusa/medusa/model/kv_cache.py` : KV-cache handling and compressed-cache support.
- `Medusa/medusa/model/triton_kernels.py` : fused verifier, KV-copy, TurboVQ/TurboQuant kernels.
- `Medusa/bench_comm_turbo.py` : benchmark harness and mode selection.
- `Medusa/train_tinylama_medusa_heads.py` : TinyLlama head training/self-distillation script.

15. References

- Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. (2024). *Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads*. ICML 2024 / arXiv:2401.10774. Local PDF: `artifacts/docs/medusa.pdf` . <https://arxiv.org/abs/2401.10774>
- Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. (2023). *Accelerating Large Language Model Decoding with Speculative Sampling*. arXiv:2302.01318. <https://arxiv.org/abs/2302.01318>
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Re, C. (2022). *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. arXiv:2205.14135. <https://arxiv.org/abs/2205.14135>
- Han, I., Kacham, P., Karbasi, A., Mirrokni, V., and Zandieh, A. (2025). *PolarQuant: Quantizing KV Caches with Polar Transformation*. arXiv:2502.02617. Local PDF: `artifacts/docs/polar_quant.pdf` . <https://arxiv.org/abs/2502.02617>
- Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M. W., Shao, Y. S., Keutzer, K., and Gholami, A. (2024). *KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization*. arXiv:2401.18079. <https://arxiv.org/abs/2401.18079>
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention*. arXiv:2309.06180. <https://arxiv.org/abs/2309.06180>
- Leviathan, Y., Kalman, M., and Matias, Y. (2023). *Fast Inference from Transformers via Speculative Decoding*. ICML 2023 / arXiv:2211.17192. <https://arxiv.org/abs/2211.17192>
- Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., and Hu, X. (2024). *KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache*. arXiv:2402.02750. <https://arxiv.org/abs/2402.02750>
- Zandieh, A., Daliri, M., and Han, I. (2024). *QJL: 1-Bit Quantized JL Transform for KV Cache Quantization with Zero Overhead*. arXiv:2406.03482. Local PDF: `artifacts/docs/qjl.pdf` . <https://arxiv.org/abs/2406.03482>
- Zandieh, A., Daliri, M., Hadian, M., and Mirrokni, V. (2025). *TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate*. arXiv:2504.19874. Local PDF: `artifacts/docs/turbo_quant.pdf` . <https://arxiv.org/abs/2504.19874>